```
import numpy as np

marks = np.array([70, 80, 65, 90, 75])

print(np.sum(marks))
print(np.mean(marks))
print(np.min(marks))
print(np.max(marks))
```

```
380
76.0
65
90
```

```
80 > 70
```

```
True
```

```
(80 < 90) and (20 > 30)
```

```
False
```

```
(80 < 90) or (20 > 30)
```

```
True
```

```
80 == 80
```

```
True
```

```
80 != 80
```

```
False
```

```
80 != 90
```

```
True
```

```
marks = np.array([70, 80, 65, 90, 75])

marks > 75
```

```
array([False,  True, False,  True, False])
```

```
# Write a code to filter in only the marks > 75.
```

```
a = [70, 80, 65, 90, 75]
b = []

for elem in a:
  if elem > 75:
    b.append(elem)

print(b)
```

```
[80, 90]
```

```
marks
```

```
array([70, 80, 65, 90, 75])
```

```
marks[[False,  True, False,  True, False]]
```

```
array([80, 90])
```

```
marks[ marks > 75 ]
```

```
array([80, 90])
```

```
temperature = np.array([
    24, 28, 31, 35, 29, 22, 37, 30
])

# temp atleast 32

temperature[ temperature >= 32 ]
```

```
array([35, 37])
```

```
prices = np.array([
    499, 799, 1299, 2499, 3999, 599
])

# Prices between 600 and 1000.

prices[(prices >= 600) and (prices <= 1000)]
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1143/4288478732.py in <cell line: 0>()
      5 # Prices between 600 and 1000.
      6
----> 7 prices[(prices >= 600) and (prices <= 1000)]

ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()
```

Next steps: Explain error

```
prices >= 600
```

```
array([False,  True,  True,  True,  True, False])
```

```
prices <= 1000
```

```
array([ True,  True, False, False, False,  True])
```

```
prices[(prices >= 600) & (prices <= 1000)]
```

```
array([799])
```

```
#prices less than 600 or greater than 1000

prices[(prices < 600) or (prices > 1000)]
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1143/1045390956.py in <cell line: 0>()
      1 #prices less than 600 or greater than 1000
      2
----> 3 prices[(prices < 600) or (prices > 1000)]

ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()
```

Next steps: Explain error

```
prices[(prices < 600) | (prices > 1000)]
```

```
array([ 499, 1299, 2499, 3999,  599])
```

```
#show prices not greater than 900

prices <= 900
```

```
array([ True,  True, False, False, False,  True])
```

```
prices[prices <= 900]
```

```
array([499, 799, 599])
```

```
prices[not (prices > 900)]
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1143/3360746840.py in <cell line: 0>()
----> 1 prices[not (prices > 900)]

ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()
```

Next steps: Explain error

```
prices[~(prices > 900)]
```

```
array([499, 799, 599])
```

```
marks = np.array([
    [72, 85, 90],
    [55, 60, 65],
    [88, 92, 95],
    [45, 50, 48]
])
```

```
# filter marks > 70
marks[marks > 70]
```

```
array([72, 85, 90, 88, 92, 95])
```

```
sales = np.array([
    [100, 200, 300],
    [400, 500, 600],
    [150, 250, 350]
])

# Find values greater than 200 only in the first row:
sales[0][sales[0] > 200]

# sales[sales[0] > 200]
```

```
array([300])
```

```
products = np.array([
    [101, 500],
    [102, 1200],
    [103, 800],
    [104, 1500]
])
```

```
# We want products prices whose price is greater than ₹1,000.
prices = products[:, 1]
```

```
prices
```

```
array([ 500, 1200,  800, 1500])
```

```
prices[prices > 1000]
```

```
array([1200, 1500])
```

Test DPP

Double-click (or enter) to edit

```
An electricity board records the monthly electricity consumption of a household for one year:

consumption = np.array([
    180, 220, 195, 250,
    280, 310, 295, 270,
    240, 210, 190, 230
])
```

```
])

Perform the following:

Find the total annual consumption.
Calculate average monthly consumption.
Find the highest and lowest consumption.
Find the month index with the highest consumption.
Extract the first 6 months using slicing.
Extract alternate months.
Find all months where consumption was greater than the annual average.
Calculate the new monthly projections if next year's consumption is expected to be 8% higher.
```

```
c = np.array([
    180, 220, 195, 250,
    280, 310, 295, 270,
    240, 210, 190, 230
])
```

```
# Find the total annual consumption.

np.sum(c)
```

```
np.int64(2870)
```

```
# Calculate average monthly consumption.
np.mean(c)
```

```
np.float64(239.16666666666666)
```

```
# Find the highest and lowest consumption.
np.min(c)
```

```
np.int64(180)
```

```
np.max(c)
```

```
np.int64(310)
```

```
# Find the month index with the highest consumption.
np.argmax(c)
```

```
np.int64(5)
```

```
# Extract the first 6 months using slicing.
c[:6]
```

```
array([180, 220, 195, 250, 280, 310])
```

```
# Extract alternate months.
c[::2]
```

```
array([180, 195, 280, 295, 240, 190])
```

```
np.mean(c)
```

```
np.float64(239.16666666666666)
```

```
# Find all months where consumption was greater than the annual average.
c[ c > np.mean(c)]
```

```
array([250, 280, 310, 295, 270, 240])
```

```
c
```

```
array([180, 220, 195, 250, 280, 310, 295, 270, 240, 210, 190, 230])
```

```
# Calculate the new monthly projections if next year's consumption is expected to be 8% higher.
```

```
  File "/tmp/ipykernel_1143/2636517555.py", line 1
    Calculate the percentage increase required if next year's consumption is expected to be 8% higher.
                                                                                                      ^
SyntaxError: unterminated string literal (detected at line 1)
```

Next steps: Explain error

```
next = c*1.08
```

```
next
```

```
array([194.4, 237.6, 210.6, 270. , 302.4, 334.8, 318.6, 291.6, 259.2,
       226.8, 205.2, 248.4])
```

```
Problem Statement

The following array contains marks of 5 students in 4 subjects:

marks = np.array([
    [72, 85, 90, 78],
    [65, 70, 75, 80],
    [88, 92, 95, 90],
    [55, 60, 58, 65],
    [78, 82, 85, 88]
])

Assume each row represents a student and each column represents a subject.

Find:

Shape and size of the array.
Marks obtained by the third student.
Marks obtained in the second subject by all students.
Average marks of every student.
Average marks of every subject.
Students whose average is above 75.
Highest mark obtained in each subject.
The complete row of the student with the highest average.
```

```
marks = np.array([
    [72, 85, 90, 78],
    [65, 70, 75, 80],
    [88, 92, 95, 90],
    [55, 60, 58, 65],
    [78, 82, 85, 88]
])
```

```
# Shape and size of the array.

marks.shape
```

```
(5, 4)
```

```
marks.size
```

```
20
```

```
# Marks obtained by the third student.
marks[2]
```

```
array([88, 92, 95, 90])
```

```
# Marks obtained in the second subject by all students.
marks[:, 1]
```

```
array([85, 70, 92, 60, 82])
```

```
# Average marks of every student.
np.mean(marks, axis = 1)
```

```
array([81.25, 72.5 , 91.25, 59.5 , 83.25])
```

```
# Average marks of every subject.
np.mean(marks, axis = 0)
```

```
array([71.6, 77.8, 80.6, 80.2])
```

```
# Students whose average is above 75.
marks
```

```
array([[72, 85, 90, 78],
       [65, 70, 75, 80],
       [88, 92, 95, 90],
       [55, 60, 58, 65],
       [78, 82, 85, 88]])
```

```
std_avg = np.mean(marks, axis = 1)
```

```
marks[ std_avg > 75 ]
```

```
array([[72, 85, 90, 78],
       [88, 92, 95, 90],
       [78, 82, 85, 88]])
```

```
# Highest mark obtained in each subject.

np.max(marks, axis = 0)
```

```
array([88, 92, 95, 90])
```

```
# The complete row of the student with the highest average.

ind = np.argmax(std_avg)
```

```
ind
```

```
np.int64(2)
```

```
marks[ind]
```

```
array([88, 92, 95, 90])
```

```
An online store has the following product prices:

prices = np.array([
    499.50, 799.90, 1299.75,
    2499.50, 3999.90, 599.25,
    899.80, 1599.40
])

Perform:

Round prices to the nearest whole number.
Find products costing more than ₹1,000.
Calculate a 15% discount only on products costing more than ₹1,000.
Calculate the average discounted price of those products.
Find the most expensive original product.
Extract the last 4 products.
Calculate the square root of the final prices and round them to 2 decimal places.
```

```
prices = np.array([
    499.50, 799.90, 1299.75,
    2499.50, 3999.90, 599.25,
    899.80, 1599.40
])
```

```
# Round prices to the nearest whole number.
np.round(prices)
```

```
array([ 500.,  800., 1300., 2500., 4000.,  599.,  900., 1599.])
```

```
# Find products costing more than ₹1,000.
prices[ prices > 1000 ]
```

```
array([1299.75, 2499.5 , 3999.9 , 1599.4 ])
```

```
# Calculate a 15% discount only on products costing more than ₹1,000.
discounted = np.round(prices[prices > 1000]*0.85)
```

```
# Calculate the average discounted price of those products.

np.mean(discounted)
```

```
np.float64(1997.25)
```

```
# Find the most expensive original product.
np.max(prices)
```

```
np.float64(3999.9)
```

```
np.argmax(prices)
```

```
np.int64(4)
```

```
# Extract the last 4 products.
prices[-4:]
```

```
array([3999.9 ,  599.25,  899.8 , 1599.4 ])
```

```
# Calculate the square root of the final prices and round them to 2 decimal places.

sqrt_prices = np.sqrt(discounted)

final = np.round(sqrt_prices, 2)
```

```
final
```

```
array([33.24, 46.1 , 58.31, 36.86])
```

```
Start coding or generate with AI.
```